

CHALLENGE 1: Rate limiter mancante

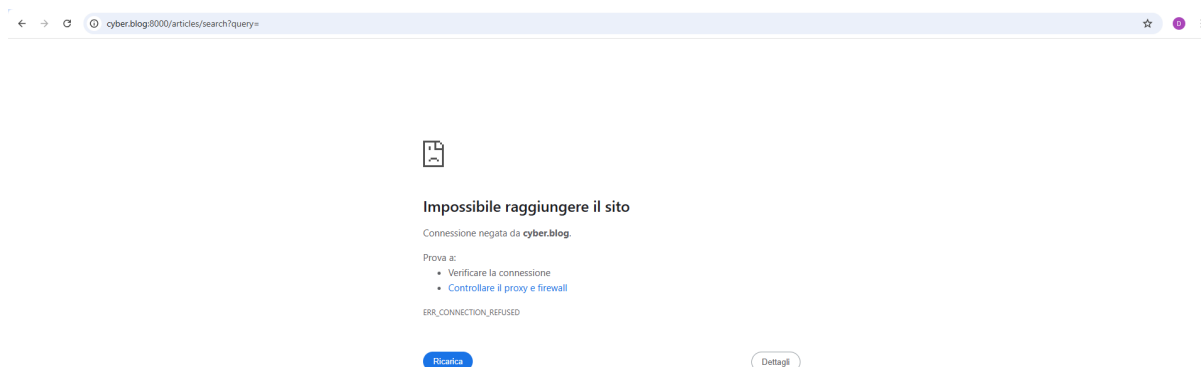
Descrizione della vulnerabilità:

La rotta /articles/search non aveva alcun rate limiting, permettendo a un attaccante di inviare centinaia di richieste in parallelo, causando un Denial of Service (DoS).

Questo avrebbe potuto rendere il sito inaccessibile per gli utenti legittimi.

```
> ▾ TERMINAL
⚠
Utente@DESKTOP-4MGBMGJ MINGW64 ~/cyber-progetti/final-project-cyber-blog/XXX-AttackTools/dos (main)
$ ./search.sh
Richiesta 22 inviata
Richiesta 23 inviata
Richiesta 24 inviata
Richiesta 25 inviata
Richiesta 26 inviata
Richiesta 27 inviata
Richiesta 28 inviata
Richiesta 29 inviata
Richiesta 30 inviata
```

Ho avuto modo di sfruttare la vulnerabilità con un script che mi ha permesso di mandare molte richieste al server in pochissimo tempo, di conseguenza il server non ha potuto reggere l'enorme traffico...



Codice implementato:

Ho aggiunto un rate limiter che limita a 5 richieste al minuto per IP sulla rotta /articles/search.

Il rate limiter è stato definito in AppServiceProvider.php per garantire una configurazione robusta.

In `app/Providers/AppServiceProvider.php`:

```
use Illuminate\Cache\RateLimiting\Limit;
use Illuminate\Support\Facades\RateLimiter;
use Illuminate\Http\Request;

// ...

public function boot(): void
{
    RateLimiter::for('search', function (Request $request) {
        return Limit::perMinute(5)->by($request->ip());
    });

    // ... resto del codice ...
}
```

In `routes/web.php`:

```
Route::get('/articles/search', [ArticleController::class, 'articleSearch'])
    ->name('articles.search')
    ->middleware('throttle:search');
```

Conclusioni:

La mitigazione ha funzionato correttamente: dopo 5 richieste, il server restituisce 429 Too Many Requests, bloccando ulteriori richieste.